

SMITH ENGINEERING

Queen's University

Department of Electrical and Computer Engineering



ELEC 490/498 Project Proposal

[Mining Software Repositories GitSkills Dataset Challenge]

Fall 2026 / Winter 2027

Submitted by Group [34]

Group member	NetID	Student number
[Noah Chung]	[20ntc2]	[Student number]
[Faiz Farook]	[21ff8]	[Student number]
[James Kilpatrick]	[21jpk5]	[20357509]
[Naman Nagia]	[21nn48]	[20357592]

Faculty supervisor(s): [Mariam Guizani]

Supervisor email: [mariam.guizani@queensu.ca]

Submission date: [07 October 2026]

Version: [1.0]

Executive Summary

AI coding agents increasingly follow *agent skills*: `SKILL.md` files of natural-language instructions that an agent loads when it judges that a task matches the skill's description. Within nine months of the format's release, public GitHub repositories held 3.8 million skill files **gitskills2027**. Skills are written mainly in prose, no compiler checks them, and they spread by being copied between repositories. Software engineering has long used metrics such as size, churn, age, clone coverage, and readability to judge source code, but it is not known whether these metrics carry over to skills. Using the GitSkills dataset from the Mining Software Repositories ([MSR](#)) 2027 Mining Challenge [2], this project asks which of these metrics have meaningful equivalents for skills and how their distributions compare with those of source code. The end product is a reproducible analysis pipeline, documented metric definitions, visualizations, and a report interpreting the findings for researchers and developers who write and reuse skills.

The pipeline loads the GitSkills tables, filters out files that are not genuine skills, runs a separate calculation script for each metric, and joins the results into a single metrics file. Each metric is adapted to skills. Size is counted in tokens, separately for the description, the body, and bundled files. Churn is split by type, because skills are more often copied and modified than edited in place. Age is measured from the first commit. Clone coverage is computed from identical content hashes. Readability scores each part of a skill with an instrument suited to it and is validated against human ratings. Performance targets include a full-dataset run within 24 h using at most 16 GB of memory, identical results on re-running, and at least 0.90 precision for cross-copy lineage links. The main milestones are a prepared and checked dataset (M1, 23 October 2026), all supported metrics calculated with initial plots (M3, 4 December 2026), and a validated, reproducible final pipeline (M6, 12 March 2027).

Every metric is first prototyped on the official 13,000-skill GitSkills sample before being scaled to the full dataset. Pilots have already run: they show that about two-thirds of skills are never edited after being added, and that skill bodies read at a median Flesch–Kincaid grade of 8.6. Testing combines unit tests against hand-calculated examples, integration tests across pipeline stages, reproducibility checks, and manual review of sample outputs. The dataset is openly licensed, all tools are open source, and computation fits on a single workstation, so the estimated total cost is **[\$0]** CAD. The team is ready to begin. Further learning is needed in large-scale data handling, text-similarity and readability measurement, and non-parametric statistics.

Contents

Executive Summary	i
Acronyms	v
1 Introduction	1
1.1 MSR Challenge and Project Motivation	1
1.2 Project Objective and Research Direction	1
1.3 Scope and Deliverables	1
2 Functional Description	2
2.1 System overview	2
2.2 Interface and performance specifications	3
2.3 Risks and safety hazards	3
3 Design and Production Approach	4
3.1 Metric Development and Analysis	4
3.2 Milestones and Division of Labor	6
3.3 Further Exploration and Contingencies	6
4 Testing and Evaluation	7
5 Resource Requirements	8
5.1 Scheduling access to resources	8
5.2 Materials and other resources	8
6 Conclusion	9
7 References	9
A Supporting Schedule and Technical Material: N/A	11
B Readability Pilot	12

List of Figures

- 1 Data flow from the GitSkills tables through the Python pipeline to the final metrics file. 2
- 2 Readability pilot on 8,314 English skills from the 13,000-skill GitSkills sample. (a) Flesch–Kincaid grade of the body prose, with the Markdown converted to sentences and as raw text. (b) Body grade under four formulas, and Flesch–Kincaid grade of the front-matter description. Boxes show the interquartile range, and whiskers extend to 1.5 times the IQR. 12

List of Tables

1	Preliminary system specifications.	3
2	Proposed technical milestones and ownership.	6
3	Proposed software verification tests.	7
4	Resource access by milestone.	8
5	Preliminary cost estimate (CAD).	8

Acronyms

MSR Mining Software Repositories

1. Introduction

1.1 MSR Challenge and Project Motivation

The Mining Software Repositories (MSR) Mining Challenge invites researchers to investigate development practices using shared software repository datasets. The 2027 challenge provides SpecMine, containing natural-language specifications for agent-assisted development, and GitSkills, containing instructions and supporting material for AI agents. Suggested research questions provide starting points for participants' own investigations [2]. Our team will use GitSkills to study how agent instructions are written, reused, and maintained in software environments. These findings could help developers and researchers identify patterns in the development and evolution of this emerging software artifact.

1.2 Project Objective and Research Direction

The project builds on MSR's question of whether established software metrics such as size, churn, age, clone coverage, and readability, have meaningful equivalents for agent skills. The team will first investigate each metric independently, then examine their distributions and relationships to guide further research and inform conclusions. Potential analyses include how instruction complexity relates to revision activity and how duplicated content is distributed across skill collections. The engineering objective is to develop a reproducible pipeline that processes and validates data, calculates metrics, and produces visualizations supporting conclusions about skill development, reuse, and maintenance.

1.3 Scope and Deliverables

The initial scope is to explore GitSkills and assess which metrics support meaningful analysis of agent instructions. Research questions and scope will be refined based on available data and preliminary results. Deliverables include documented metric definitions, reproducible analysis code, visualizations, and a report interpreting the findings. Data availability will determine which analyses can be completed, and the report will explain the limitations of the measurements and conclusions.

2. Functional Description

2.1 System overview

Since this is a dataset analysis project, the system itself is fairly simple, as shown in Figure 1. GitSkills is distributed as a single SQLite database, with a Parquet copy of each table, and holds four tables: `artifacts`, `artifact_siblings`, `repos`, and `mining_runs`. The fields we need are queried from these tables into Python, where we rely on common data-analysis libraries: `pandas` and `NumPy` for data handling, `DuckDB` and `PyArrow` for querying the Parquet files, `tiktoken` for token counts, `textstat` for readability formulas, `datasketch` for MinHash similarity, `SciPy` and `powerlaw` for statistics, and `Matplotlib` for plots. A pre-processing script first transforms it for the metric scripts and filters out files that are not genuine skills (`filename`, `location_class`, `frontmatter_valid`). The cleaned data then passes to a separate calculation script for each metric, whose methods are described in Section 3. Finally, an output transformation script joins the results into a single metrics file used for analysis, plotting, and validation. The only input from outside the dataset is a small sample of cloned GitHub repositories used to validate churn.

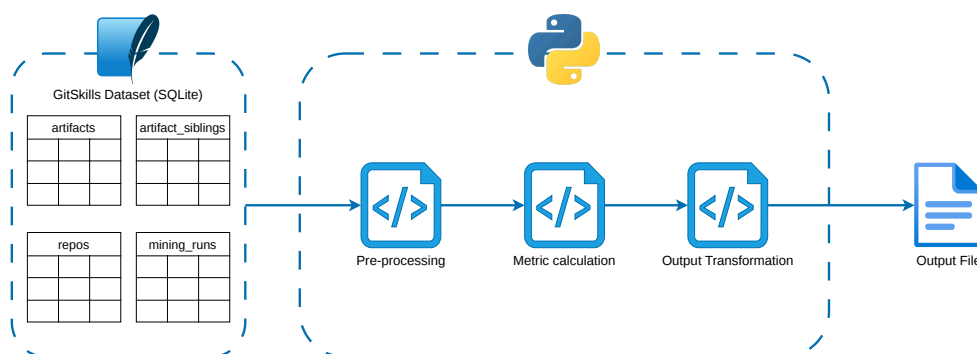


Figure 1: Data flow from the GitSkills tables through the Python pipeline to the final metrics file.

2.2 Interface and performance specifications

Table 1: Preliminary system specifications.

ID	Requirement	Target	Tolerance
IF-01	All four GitSkills tables load into Python with their documented columns	4 of 4 tables	0 missing columns
IF-02	Scripts pass data keyed by <code>file_sha</code> , with no rows lost between stages	100% of rows kept	0 rows
IF-03	Output file holds one row per distinct skill, in Parquet and CSV	1 row per skill	0 duplicates
PF-01	Full-dataset run time on one workstation	≤ 24 h	+12 h
PF-02	Peak memory use, kept low through batching	≤ 16 GB	+4 GB
PF-03	Re-running the pipeline reproduces the same metrics	Identical output	0 differences
PF-04	Precision of cross-copy lineage links on the hand-labelled sample	≥ 0.90	−0.05
PF-05	Spearman correlation between Flesch–Kincaid grade and the mean human rating on the 100-skill readability sample	$ \rho \geq 0.5$	−0.1
PF-06	Agreement between the two readability raters (Krippendorff’s α krippendorff2004)	≥ 0.80	−0.13

2.3 Risks and safety hazards

The project is purely computational, so it carries no physical, health, or safety hazards. The main technical risk is the dataset’s size (about 44 GB), which is managed by reading the per-table Parquet files in batches rather than loading everything into memory. Since the data comes from public GitHub repositories with commit authors already anonymized, we will not attempt to identify individuals, and we will respect each repository’s license when quoting skill content.

3. Design and Production Approach

3.1 Metric Development and Analysis

The following outlines the proposed methods for measuring size, churn, age, clone coverage, and readability, including the data requirements, calculation procedures, and validation approach for each metric. Each metric follows four rough stages: it is prototyped on the official 13,000-skill GitSkills sample, calibrated, scaled to the full dataset, and validated.

Size. Size is classically measured in lines, but that doesn't translate well to skills. A skill mixes plain-language instructions with code blocks, and a line of prose and a line of code carry very different amounts of information; in our pilot, about 12% of a typical skill's body was code. Instead, we measure size in tokens, the units an agent actually reads text in and the units its context window is limited by. We split that count across the three parts of a skill, since an agent loads each at a different time: the *description*, read for every installed skill to decide which one to use; the *body*, loaded only once a skill is chosen; and the *bundle*, the scripts and reference files the body points to. Measuring each part separately shows not just how big a skill is, but where that size costs the agent. Lines and words are still recorded alongside tokens, so skills can be compared directly with the code bundled next to them.

Churn. Classically, churn is the number of lines added plus lines removed between commits, and it shows how much a file changes as it develops and what patterns those changes follow. Since lines don't fit skills well, churn needs adapting too, and the data adds a second complication: GitSkills records how often and when a skill was committed, but not what changed, and about two-thirds of skills in our pilot were never edited after being added. What skills tend to do instead is get copied and then modified for a new project, so we split churn into three types. *Activity churn* uses the recorded commit counts and dates to measure how often, and for how long, a skill is maintained. *Cross-copy churn* links each skill to the copy it most likely came from using text similarity (MinHash), with the cut-off for "same skill" checked by hand, and counts the tokens added and removed along the way; this is where we expect the most useful results, since it captures how popular templates get adapted for uses like design or security. *Within-file churn* is classical churn computed from the full git history of a sample of edited skills, and confirms that the other two measures reflect real change.

Age. Age is usually measured as the amount of time since the file was introduced to the repository. This method of measurement will work well for skills. The GitSkills artifact subset has fields: `first_commit_at` and `last_commit_at`. The team will calculate the difference between `first_commit_at` and the dataset collection date, then convert this difference to days to create the Age metric.

Clone Coverage. Clone coverage measures how much content in a dataset is duplicated. In the GitSkills artifacts subset, there is a field called `content_hash`. Clones can be identified by checking to see if they have the same `content_hash`. To calculate the clone coverage percentage, the team will group skills by `content_hash` and count the number of cloned skill occurrences. The number of cloned skill occurrences will then be divided by the total number of skill occurrences then multiplied by 100 to get the clone coverage percentage.

$$\text{Clone Coverage} = \frac{\text{Number of cloned skill occurrences}}{\text{Total number of skill occurrences}} \times 100.$$

Readability. Readability is classically measured in two separate ways. Formulas such as the Flesch–Kincaid grade level score English prose by sentence and word length **flesch1948**, **kincaid1975**, while code readability models score code from features such as line count and token entropy **buse2010readability**, **posnett2011simpler**. Neither fits a skill on its own, so we split each skill into the same three parts used for size and score each part with the instrument designed for it. The body’s prose and the description are scored with the Flesch–Kincaid grade, with three related formulas as a cross-check. The body’s code blocks are scored with Posnett’s model, which is also applied, in equal-length snippets, to the scripts bundled next to the skill, so skill code and ordinary code are compared on the same scale. Before scoring, the Markdown has to be turned into real sentences. Otherwise each bullet list counts as one long sentence, and in our pilot on the 13,000-skill sample this raised the median skill’s grade by 3.4 levels (Appendix B). The formulas only work for English, so non-English skills (about 21% of the pilot) are counted but not scored. In the pilot, the median skill body read at grade 8.6, while descriptions read at grade 12.7. To validate, two team members will independently rate 100 skills, stratified by grade, on a five-point scale. The formula is accepted if the raters agree with each other (PF-06) and its scores track their average rating (PF-05). If it fails, readability will be reported only as a measure of text complexity, not of quality.

3.2 Milestones and Division of Labor

Progress will be reviewed intermittently during Friday team meetings. Each member will be required to lead at least one milestone, coordinating the work and overseeing its timely completion. Responsibility for implementation and delivery will be shared across the team.

Table 2: Proposed technical milestones and ownership.

ID	Milestone	Due date	Lead
M1	Dataset prepared and sample records checked.	Oct. 23, 2026	Naman
M2	Initial metric calculations checked against reference examples.	Nov. 13, 2026	James
M3	Supported metrics calculated; initial tables and plots generated.	Dec. 4, 2026	Faiz
M4	Results reviewed; focused research questions documented.	Jan. 29, 2027	Noah
M5	Follow-up investigations evaluated; feasible extensions tested.	Feb. 26, 2027	Naman
M6	Final pipeline validated and results reproduced.	Mar. 12, 2027	James

3.3 Further Exploration and Contingencies

Initial findings will guide M5, which may examine instruction structure, bundled scripts and references, or reuse patterns. Additional analyses will depend on relevance, data availability, and remaining time.

Missing data or unreliable metrics will prompt revised methods or a narrower scope. Processing limits will be managed through batching or documented sampling to control time and costs. If delays threaten validation, tasks will be reassigned and optional investigations deferred. Resulting limitations will be documented.

4. Testing and Evaluation

Testing will assess the correctness and reproducibility of the analysis pipeline, from data preparation through metric calculation and visualization. Unit tests will use controlled examples with known expected results, while integration tests will check that the processing stages work together correctly. Table 3 summarizes the proposed tests and completion criteria.

Table 3: Proposed software verification tests.

Component	Proposed test	Expected evidence of success
Data preparation	Check sample records containing missing fields, duplicate content, and unusual formatting.	Records are retained, excluded, or flagged according to documented rules.
Metric calculations	Compare implemented calculations with manually calculated or independently verified examples.	Results match expected values within defined numerical tolerances.
Readability validity	Two members independently rate 100 skills, stratified by grade; ratings are compared with each other and with the formula scores.	Raters agree and formula scores track the mean rating (PF-05, PF-06).
Pipeline integration	Process a controlled sample through all analysis stages.	Record identifiers and calculated values remain consistent across intermediate and final outputs.
Reproducibility	Repeat the analysis using the same dataset, configuration, and software versions.	Tables and plotted values agree across repeated runs within defined numerical tolerances.

Evaluation will also consider whether the selected metrics and visualizations address the refined research questions. Sample outputs will be reviewed manually to confirm that labels, units, and interpretations agree with the underlying data. Any unsupported measurements or data limitations will be documented.

Testing will use the team’s development environment, a unit-testing framework, and controlled samples from the selected dataset. Identified defects will be corrected and affected tests rerun before the final analysis is accepted.

5. Resource Requirements

5.1 Scheduling access to resources

Table 4: Resource access by milestone.

Milestone	Resource / source	Access date / lead time
M1	GitSkills 13,000-skill sample, Python, GitHub repository, VS Code/Jupyter, data-processing libraries	Immediate, publicly available and open source
M2–M3	Full GitSkills dataset, local storage, Python analysis environment, source-code comparison dataset	Required before full-scale analysis, sufficient storage required
M4–M5	Statistical, text-analysis, tokenization, and visualization libraries, processed metric files	Required during metric calculation, validation, and full-dataset analysis
M6	GitHub repository, processed results, plots, validation and reproducibility scripts	Continuous access through final validation and reporting

5.2 Materials and other resources

Table 5: Preliminary cost estimate (CAD).

Item / source	Qty.	Unit cost	Total
GitSkills dataset and official sample	1	\$0	\$0
Python, Git, VS Code/Jupyter, and open-source analysis libraries	1 set	\$0	\$0
GitHub repository and version-control access	1	\$0	\$0
Local data storage	1	\$0	\$0
Estimated total			\$0

6. Conclusion

This project aims to examine size, churn, age, clone coverage, and readability within AI agent skills using the GitSkills dataset. The team plans to study how these skills are developed and maintained, and whether traditional software engineering metrics can be meaningfully applied to them. The final system will be a Python analysis pipeline that processes the dataset and calculates metrics based on the available fields.

Development will follow six milestones (M1–M6). M1 focuses on dataset preparation and verification of sample records. Next, M2 involves the initial metric calculations and checking them against reference examples. Once the methods have been established, the team will calculate the supported metrics across the dataset and generate preliminary tables and visualizations (M3). This is followed by M4, which involves reviewing results and documenting focused research questions. Based on these findings, the team will investigate follow-up questions and potential extensions during M5. Finally, the pipeline will undergo validation and reproducibility testing in M6. The pipeline will be designed to process the full dataset within 24 hours while using no more than 16 GB of memory. Testing will include unit tests, integration tests, and reproducibility checks to ensure the calculated metrics are accurate and consistent. Since the project relies on publicly available data and open-source tools, the estimated total cost is \$0 CAD.

The expected outcome of the project is to identify and document meaningful metrics that can help developers and researchers better understand AI agent skills and how they evolve. The full scope of the project will depend on the results of preliminary analysis, as focused research questions will be established during M4 and investigated further during M5. Although this makes part of the project's scope open-ended, the team has established a clear development plan, assigned milestones, and identified the resources needed for success. With initial pilot analyses already completed, the team is prepared to move forward with implementation and validation.

7. References

- [1] International Council on Systems Engineering, *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 4th ed., D. D. Walden, G. J. Roedler, K. J. Forsberg, R. D. Hamelin, and T. M. Shortell, Eds. Hoboken, NJ: John Wiley & Sons, 2015, ISBN: 978-1-118-99940-0.

- [2] *MSR 2027 Mining Challenge: Call for Mining Challenge Papers*, <https://conf.researchr.org/track/msr-2027/msr-2027-mining-challenge>, Accessed: Oct. 7, 2026, International Conference on Mining Software Repositories.

A. Supporting Schedule and Technical Material: N/A

B. Readability Pilot

We ran the readability method on the official 13,000-skill GitSkills sample to check that it is feasible and to choose the preprocessing. Of the 13,000 distinct skills, 237 were not named exactly `SKILL.md`, 1,566 had invalid front matter, 555 were too short (a body under 300 characters or fewer than 30 words of prose), and 2,328 were not in English. This left 8,314 skills from 5,567 repositories. Figure 2a shows why the Markdown must be converted into sentences before scoring: on raw text, each bullet list counts as one long sentence, and the median grade rises from 8.6 to 12.2. Figure 2b shows that skill sentences are short (a median of 9 words), but formulas that weight word length, such as Coleman–Liau, place the same text near grade 13, which reflects technical vocabulary. Descriptions are usually a single long sentence, and they read about four grades harder than the bodies. The pilot code uses textstat `textstat`; whole-sample scoring took about 6 ms per skill on one laptop core, or roughly 3 h for the full 1.88 million distinct skills.

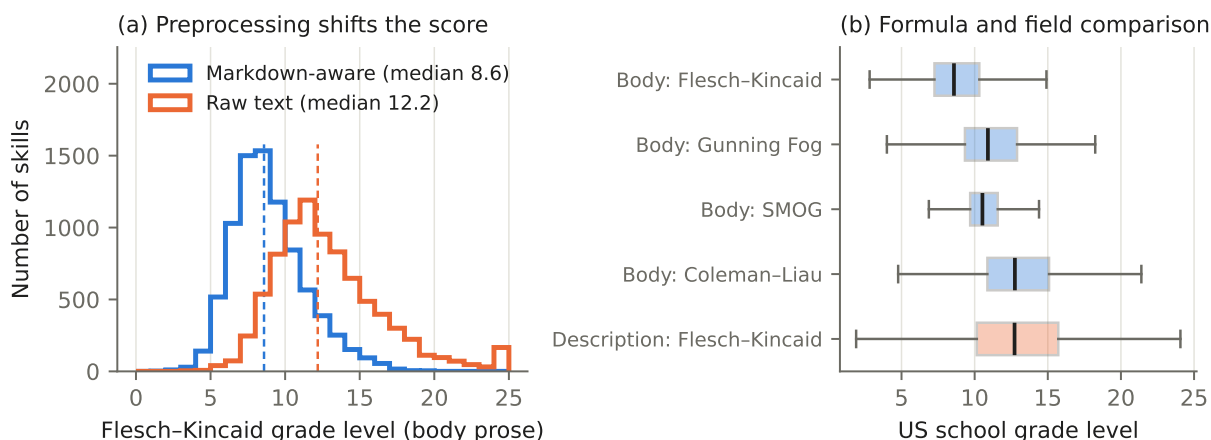


Figure 2: Readability pilot on 8,314 English skills from the 13,000-skill GitSkills sample. (a) Flesch–Kincaid grade of the body prose, with the Markdown converted to sentences and as raw text. (b) Body grade under four formulas, and Flesch–Kincaid grade of the front-matter description. Boxes show the interquartile range, and whiskers extend to 1.5 times the IQR.